1- • The attacker, acting as a restricted staff member on the target store (`Store A`) who lacks file upload permissions, creates a separate, attacker-controlled store (`Store B`) where they have full admin rights. They upload their target file to `Store B`.
• **Data Extraction:** The attacker extracts the backend identifiers for the uploaded file from `Store B` (specifically the `absoluteKey`, `key`, and the CDN `path`).
• **API Exploitation:** The attacker logs into `Store A` with their restricted staff account and targets an undocumented internal GraphQL mutation called `fileCopy`.
• **Execution:** The attacker sends a `POST` request to `/admin/internal/web/graphql/core` on `Store A`, executing the `fileCopy` mutation and passing the extracted file identifiers from `Store B` as variables.

2- • The attacker sends an authenticated request to the GitLab Search API (`/api/v4/projects/<id>/search?scope=blobs`).
• **Payload Delivery:** The attacker provides a generic search string and injects the specific git flag `--no-index` into the `ref` parameter.
• **Backend Execution:** The GitLab backend (Gitaly) unsafely concatenates the user-controlled `ref` parameter directly into an underlying OS command (`git grep`).
• **Flag Exploitation:** The injected `--no-index` flag explicitly instructs the `git grep` command to ignore the git repository index and instead search the current working directory of the host filesystem (in this case, `/var/opt/gitlab/gitaly`).
• **Data Exfiltration:** Because the search is executed on the local filesystem, `git grep` reads internal server files (like `config.toml`). The API captures this command output and reflects the contents of the arbitrary files back to the attacker in the HTTP JSON response.

3- By intercepting the web application's traffic using a proxy and opening an existing chat, the attacker triggers a legitimate POST request to the backend API (/mwebapi.phtml?SERVER_OPEN_CHAT). Although the frontend UI intentionally hides the message "read" status to protect user privacy—displaying only a "delivered" status—the backend over-fetches and leaks this sensitive information. The attacker can simply inspect the raw JSON response to locate the read boolean key nested within the chat_messages array, completely bypassing the intended frontend privacy restrictions to reveal the exact read status of all their sent messages.

4- By authenticating with a valid account and intercepting a standard profile request, an attacker can modify the username parameter in the GET /api/v1/users/{username} API endpoint to target any arbitrary user on the platform. Because the backend lacks proper object-level authorization checks and over-fetches user object attributes, the API

response inadvertently leaks sensitive internal metadata—specifically the is_email_confirmed and is_email_public boolean fields. This exposes the target's email privacy configurations and allows the attacker to silently enumerate verified, active accounts, even when the actual email address is explicitly hidden from the public UI.

5- an attacker triggers a **sensitive data leak** within the application's error handling logic. Instead of a generic failure message, the system reflects the **exact token string** back into the UI via a client-side popup. This improper handling of credentials allows the **plain-text exposure** of sensitive input, potentially revealing secret keys or partially correct authentication data that should remain masked even during a validation failure. It's a classic case of the error message being a bit too honest for its own good.

6- By inviting a secondary user with a restricted "Moderator" role through the Shared Access feature and switching to the parent account's context, an attacker can bypass intended permission boundaries to exfiltrate sensitive administrative credentials. While the standard API correctly enforces access control and rejects unauthorized profile requests with a 401 error, the platform API endpoint at https://platform.streamlabs.com/api/v1/s/user/me fails to validate the delegated user's scope or ownership. By directly querying this vulnerable endpoint, the restricted moderator successfully retrieves the parent user's private metadata, including their email address and, most critically, their active JWT token, which effectively grants the attacker full unauthorized access to the parent's account beyond their assigned moderator privileges.

7- An authenticated attacker exploits a blind NoSQL injection vulnerability in the Rocket.Chat users.list API by passing an unsanitized MongoDB $where operator into the query URL parameter to execute arbitrary JavaScript on the database. By crafting a boolean oracle that uses regular expressions to test field values character-by-character, the attacker systematically leaks sensitive administrative metadata, including emails, 2FA secrets, and password reset tokens. After triggering a legitimate password reset for a targeted administrator and using the leaked token to hijack the account, the attacker leverages their new administrative privileges to create an incoming webhook integrated with a server-side script. Because these scripts execute directly on the server without a security boundary, the attacker achieves full Remote Code Execution (RCE), allowing them to drop into an interactive shell as the rocketchat user.

8- An attacker exploits a memory exhaustion vulnerability in Mattermost by uploading a **31-byte GIF** with maliciously crafted header dimensions through the **UploadData** API. The vulnerability occurs because the backend logic calls **gif.DecodeAll** via the **GetInfoForBytes** function without first enforcing dimension limits or using the pre-processing routines found in other file-upload routes. Upon receiving the file, the server attempts to allocate a massive memory buffer (exceeding **4GB**) based on the extreme width and height values defined in the malicious GIF header. This results in an immediate **Out-of-Memory (OOM)** event that causes the Docker container or the server process to be killed, leading to a complete Denial of Service.

9- An authenticated attacker with the commonly assigned **view-d-room** permission exploits an insecure implementation of the **/api/v1/users.list** REST endpoint by passing a raw JSON object through the **query** URL parameter. Because the backend unsafely maps this client-side input directly to the **Users.find()** database operation without proper validation or field-level filtering, it essentially enables a **NoSQL injection**. By crafting a specific query—such as **{"roles":"admin"}**—the attacker can force the server to exfiltrate sensitive metadata, including private emails and role assignments, for every account in the database. This unauthorized access to the entire user collection bypasses intended administrative boundaries and provides the necessary reconnaissance data to facilitate high-impact follow-up attacks, such as targeted password brute-forcing against administrative accounts.

10- An attacker exploits a missing rate limit on the /accounts/email/ endpoint by automating POST requests to the platform's email functionality. Because the server fails to enforce throttling or request quotas, an attacker can programmatically resubmit the form to flood any arbitrary email address with a high volume of unwanted messages. This lack of control facilitates large-scale email bombing and can lead to resource exhaustion or a localized Denial of Service (DoS) by overwhelming the mail server's outgoing queue.